EPITECH PROJECT - NETWORKING / SECURITY

My_sniffer

Build your own cross-platform network packet sniffer

DURATION	5 weeks	TEAM	Individual
LANGUAGE	C / C++		

Foreword

Every packet that leaves or reaches your machine goes through a stack of protocols you never look at. Being able to open that stack up, to actually see, byte by byte, what your network card is doing, is a foundational security and networking skill. In this project, you will build your own network sniffer, from raw packet capture up to behavioral analysis.

General Instructions

- The project must be written in C or C++.
- The use of any networking library (raw capture, packet parsing, protocol decoding, etc.) is forbidden. Everything related to network capture and parsing must be implemented by yourself.
- Libraries unrelated to networking (logging, event loop, testing, CLI parsing, etc.) are allowed.
- If in doubt about whether a library is allowed, ask the teaching team before using it.
- Your project must compile without errors, warnings, or crashes.
- Any function crashing (segfault, bus error, double free, unhandled exception, etc.) will cause you to lose the maximum grade for the corresponding part.

- You are free to use whatever build system you want (CMake, Makefile, etc.), as long as everything needed to build, install and run your project on each supported platform is present in your repository.
- This project is not a school exercise, it is a product. You are expected to think like an engineer shipping a tool to real users, not like a student handing in homework: build process, installation, documentation and versioning matter as much as the code itself.
- Capturing raw network traffic requires elevated privileges on most systems, this is expected, document how to run your program.

Mandatory Part

Step 0: Function helpers

You will need to read/parse IP addresses and other meta information in binary. You must build your own predefined functions that let you process/convert this information.

Step 1: Build your own network packet viewer

You must find a way to capture all packets going through your own network interface.

> **Constraint:** your project must be cross-platform (at least Linux and one other OS of your choice). It is up to you to figure out how low-level network capture works on each platform, and what consequences that has on your architecture.

Protocols to support (minimum requirement): Ethernet, IPv4, TCP and UDP.

Step 2: Filter/search

You must be able, with your program, to filter/search through the visualized packets based on rules you define yourself.

Step 3: Analysis

You must be able to recognize 'strange' behaviors: a lot of requests to a given service, a lot of error returns from another, etc.

Check for unexpected traffic spikes, unauthorized connections to foreign IP addresses, unknown protocols running on standard ports, or regular, timed beaconing intervals.

Think of what you would look for to detect another sniffer on the network, or basic traffic anomalies used for troubleshooting.

Step 4: Packaging and deployment

A sniffer nobody can install is a sniffer nobody uses. You must provide everything needed to build, install and run your product on each platform you support, not just source code that happens to compile on your own machine.

This includes, at least:

- A build system (of your choice) with a proper install target.
- Build and/or install scripts adapted to each supported platform (e.g. handling platform-specific dependencies or privileges needed for packet capture).
- Versioning of your releases (tags, changelog, whatever convention you choose, as long as it is consistent).
- Documentation covering: how to build, how to install, how to run, and what privileges/configuration are required.

You are expected to take care of:

- capture modules (cross-platform)
- packet parsing
- packet filtering
- packet analysis
- logging and storage
- build, packaging and deployment

Bonus Part

Bonus will only be evaluated if the mandatory part is perfect.

This means: if you fail to validate every mandatory requirement, your bonus part will not be evaluated, no matter how good it is.

- Support for additional protocols: ARP, ICMP, IPv6.
- Recognition of common application-layer protocols (DNS, HTTP).
- Any relevant addition to the analysis engine (e.g. detecting another sniffer running on the network).
- A GUI and/or TUI on top of your sniffer.

Submission and Peer-Evaluation

This project will be evaluated by humans only. Do not hesitate to double check your understanding of the instructions before defense.

Only the work in your repository will be evaluated. Any work outside of it will not be taken into account.

Resources

- Network stacks, ethernet frame? https://www.youtube.com/watch?v=PG9oKZdFb7w
- Protocols: https://www.youtube.com/watch?v=E5bSumTAHZE
- Packet sniffer basics for network troubleshooting: https://www.youtube.com/watch?v=muOy-9RVrTQ
- https://www.youtube.com/watch?v=h6bPC7H6j48 (edgy hacker video)
- How to Detect Sniffer in Your Network? - GeeksforGeeks